In [44]:
```
import numpy as np
np.random.randint(1,5,3)
```

Out[44]:
```
array([3, 4, 4], dtype=int32)
```

In [2]:
```
np.random.randint(5,2,3)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[2], line 1
----> 1 np.random.randint(5,2,3)

File numpy/random/mtrand.pyx:794, in numpy.random.mtrand.RandomState.randint()

File numpy/random/_bounded_integers.pyx:1425, in numpy.random._bounded_integers._
rand_int32()

ValueError: low >= high
```

In [ ]:
```
# randn vs randint
# randn(3) --> it will generate 3 random float numbers
# randint(3) --> it will generate 3 integers 0,1,2
```

In [3]:
```
np.random.randint(-5,2,3)
```

Out[3]:
```
array([-2,  1, -1], dtype=int32)
```

In [4]:
```
np.random.randint(10,40,(10,10)) #(10,10) is matrix size
```

Out[4]:
```
array([[16, 39, 19, 16, 19, 19, 13, 13, 34, 27],
       [34, 32, 32, 16, 18, 25, 16, 39, 26, 22],
       [27, 24, 16, 34, 17, 17, 25, 26, 23, 12],
       [15, 32, 26, 28, 23, 11, 18, 11, 20, 33],
       [37, 11, 31, 36, 26, 24, 30, 13, 14, 33],
       [30, 19, 27, 28, 24, 12, 27, 14, 33, 37],
       [14, 37, 14, 24, 39, 12, 30, 16, 13, 13],
       [29, 24, 37, 21, 18, 10, 37, 27, 11, 17],
       [30, 12, 22, 27, 38, 28, 29, 31, 16, 19],
       [27, 36, 19, 20, 26, 23, 33, 11, 24, 28]], dtype=int32)
```

In [5]:
```
ri = np.random.randint(10,40,(10,10))
ri
```

Out[5]:
```
array([[11, 13, 28, 11, 32, 37, 20, 14, 20, 34],
       [36, 10, 37, 24, 12, 10, 39, 33, 23, 24],
       [17, 29, 20, 26, 24, 20, 37, 16, 37, 21],
       [16, 15, 32, 34, 19, 27, 24, 35, 28, 16],
       [39, 29, 12, 12, 25, 24, 30, 39, 22, 25],
       [21, 29, 11, 34, 36, 19, 29, 19, 29, 34],
       [23, 30, 14, 28, 17, 14, 39, 26, 34, 34],
       [12, 24, 33, 23, 24, 21, 15, 19, 24, 35],
       [33, 23, 19, 26, 30, 15, 16, 24, 13, 16],
       [26, 27, 28, 17, 23, 29, 36, 21, 17, 32]], dtype=int32)
```

In [6]:
```
len(ri) # It will only display the rows
```

Out[6]:
```
10
```

In [ ]:
```
ri.shape # It will dsiplay both col and rows
```

SLICING IN NUMPY

In [7]:
```
ri[:]
```

Out[7]:
```
array([[11, 13, 28, 11, 32, 37, 20, 14, 20, 34],
       [36, 10, 37, 24, 12, 10, 39, 33, 23, 24],
       [17, 29, 20, 26, 24, 20, 37, 16, 37, 21],
       [16, 15, 32, 34, 19, 27, 24, 35, 28, 16],
       [39, 29, 12, 12, 25, 24, 30, 39, 22, 25],
       [21, 29, 11, 34, 36, 19, 29, 19, 29, 34],
       [23, 30, 14, 28, 17, 14, 39, 26, 34, 34],
       [12, 24, 33, 23, 24, 21, 15, 19, 24, 35],
       [33, 23, 19, 26, 30, 15, 16, 24, 13, 16],
       [26, 27, 28, 17, 23, 29, 36, 21, 17, 32]], dtype=int32)
```

In [8]:
```
ri[1] # it will prints first row only
```

Out[8]:
```
array([36, 10, 37, 24, 12, 10, 39, 33, 23, 24], dtype=int32)
```

In [9]:
```
ri[0:3] #it will prints from 0th row to 2nd row
```

Out[9]:
```
array([[11, 13, 28, 11, 32, 37, 20, 14, 20, 34],
       [36, 10, 37, 24, 12, 10, 39, 33, 23, 24],
       [17, 29, 20, 26, 24, 20, 37, 16, 37, 21]], dtype=int32)
```

In [10]:
```
ri_1= np.random.randint(20,50,(10,10))
ri_1
```

Out[10]:
```
array([[38, 45, 42, 31, 38, 48, 32, 36, 24, 41],
       [38, 41, 44, 30, 28, 40, 47, 23, 42, 30],
       [29, 30, 26, 42, 32, 30, 38, 46, 47, 23],
       [28, 42, 35, 24, 47, 28, 40, 35, 33, 42],
       [29, 42, 21, 30, 43, 31, 49, 35, 35, 42],
       [22, 36, 36, 28, 43, 28, 49, 22, 42, 46],
       [42, 45, 31, 25, 44, 20, 31, 41, 22, 31],
       [25, 36, 34, 41, 22, 21, 35, 37, 49, 44],
       [43, 29, 33, 28, 47, 39, 34, 43, 44, 20],
       [23, 43, 20, 23, 46, 29, 35, 24, 35, 34]], dtype=int32)
```

In [11]:
```
print(ri_1) # It will disappearing the dtype= int32 in output
```

```
[[38 45 42 31 38 48 32 36 24 41]
 [38 41 44 30 28 40 47 23 42 30]
 [29 30 26 42 32 30 38 46 47 23]
 [28 42 35 24 47 28 40 35 33 42]
 [29 42 21 30 43 31 49 35 35 42]
 [22 36 36 28 43 28 49 22 42 46]
 [42 45 31 25 44 20 31 41 22 31]
 [25 36 34 41 22 21 35 37 49 44]
 [43 29 33 28 47 39 34 43 44 20]
 [23 43 20 23 46 29 35 24 35 34]]
```

In [12]:
```
ri_1[3:8]
```

Out[12]:
```
array([[28, 42, 35, 24, 47, 28, 40, 35, 33, 42],
       [29, 42, 21, 30, 43, 31, 49, 35, 35, 42],
       [22, 36, 36, 28, 43, 28, 49, 22, 42, 46],
       [42, 45, 31, 25, 44, 20, 31, 41, 22, 31],
       [25, 36, 34, 41, 22, 21, 35, 37, 49, 44]], dtype=int32)
```

In [14]:
```
ri_1[3,8] # --> here 3 represnts the row and 8 represents the element in that ro
```

Out[14]:
```
np.int32(33)
```

In [15]:
```
ri_1[-5]
```

Out[15]:
```
array([22, 36, 36, 28, 43, 28, 49, 22, 42, 46], dtype=int32)
```

In [16]:
```
ri_1[-5,-5]
```

Out[16]:
```
np.int32(28)
```

In [17]:
```
ri_1[::3]
```

Out[17]:
```
array([[38, 45, 42, 31, 38, 48, 32, 36, 24, 41],
       [28, 42, 35, 24, 47, 28, 40, 35, 33, 42],
       [42, 45, 31, 25, 44, 20, 31, 41, 22, 31],
       [23, 43, 20, 23, 46, 29, 35, 24, 35, 34]], dtype=int32)
```

In [22]:
```
ri_1[::-1]
```

Out[22]:
```
array([[26, 25, 23, 43, 49, 49, 39, 28, 49, 37],
       [40, 28, 39, 37, 27, 49, 20, 38, 30, 24],
       [28, 39, 38, 29, 41, 22, 35, 35, 27, 25],
       [28, 20, 35, 36, 27, 30, 36, 38, 34, 46],
       [47, 38, 31, 24, 47, 48, 48, 26, 30, 26],
       [46, 26, 44, 36, 27, 43, 35, 29, 29, 35],
       [21, 20, 33, 44, 40, 26, 43, 32, 44, 26],
       [38, 47, 38, 40, 46, 47, 27, 34, 38, 49],
       [47, 34, 39, 44, 45, 24, 45, 22, 25, 49],
       [25, 25, 20, 42, 35, 43, 36, 21, 49, 22]], dtype=int32)
```

In [23]:
```
ri_1[::-5]
```

Out[23]:
```
array([[26, 25, 23, 43, 49, 49, 39, 28, 49, 37],
       [46, 26, 44, 36, 27, 43, 35, 29, 29, 35]], dtype=int32)
```

In [19]:
```
print(ri_1)
```

```
[[38 45 42 31 38 48 32 36 24 41]
 [38 41 44 30 28 40 47 23 42 30]
 [29 30 26 42 32 30 38 46 47 23]
 [28 42 35 24 47 28 40 35 33 42]
 [29 42 21 30 43 31 49 35 35 42]
 [22 36 36 28 43 28 49 22 42 46]
 [42 45 31 25 44 20 31 41 22 31]
 [25 36 34 41 22 21 35 37 49 44]
 [43 29 33 28 47 39 34 43 44 20]
 [23 43 20 23 46 29 35 24 35 34]]
```

In [18]:
```
print(ri_1[2])
print(ri_1[:,2])
```

```
[29 30 26 42 32 30 38 46 47 23]
[42 44 26 35 21 36 31 34 33 20]
```

In [20]:
```
ri_1[:,8] # Here 8 represents the column
```

Out[20]:
```
array([24, 42, 47, 33, 35, 42, 22, 49, 44, 35], dtype=int32)
```

In [21]:
```
ri_1[:,-8] # " : " n-1 is applicable " , " n-1 is not applicable
```

Out[21]:
```
array([42, 44, 26, 35, 21, 36, 31, 34, 33, 20], dtype=int32)
```

In [22]:
```
print(ri_1)
```

```
[[38 45 42 31 38 48 32 36 24 41]
 [38 41 44 30 28 40 47 23 42 30]
 [29 30 26 42 32 30 38 46 47 23]
 [28 42 35 24 47 28 40 35 33 42]
 [29 42 21 30 43 31 49 35 35 42]
 [22 36 36 28 43 28 49 22 42 46]
 [42 45 31 25 44 20 31 41 22 31]
 [25 36 34 41 22 21 35 37 49 44]
 [43 29 33 28 47 39 34 43 44 20]
 [23 43 20 23 46 29 35 24 35 34]]
```

In [23]:
```
print(ri_1[0:3,0:3]) # first 0:3 represents row and next 0:3 represents column
```

```
[[38 45 42]
 [38 41 44]
 [29 30 26]]
```

In [24]:
```
print(ri_1[2:6,3:8]) # row, col
```

```
[[42 32 30 38 46]
 [24 47 28 40 35]
 [30 43 31 49 35]
 [28 43 28 49 22]]
```

In [25]:
```
np.arange(1,13)
```

Out[25]:
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
```

In [29]:
```
np.arange(1,13).reshape(3,4) # from 1-12 numbers are reshapeing into the 3 x 4 m
```

Out[29]:
```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

In [30]:
```
np.arange(1,13).reshape(5,4) # Here we get error because we have 12 elements we
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[30], line 1
----> 1 np.arange(1,13).reshape(5,4) # Here we get error because we have 12 eleme
nts we cant arrnage them into 5 X 4 matrix

ValueError: cannot reshape array of size 12 into shape (5,4)
```

NUMPY MASKING OR FILTERS

In [32]:
```
mat = np.arange(0,100).reshape(10,10)
print(mat)
```

```
[[ 0  1  2  3  4  5  6  7  8  9]
 [10 11 12 13 14 15 16 17 18 19]
 [20 21 22 23 24 25 26 27 28 29]
 [30 31 32 33 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47 48 49]
 [50 51 52 53 54 55 56 57 58 59]
 [60 61 62 63 64 65 66 67 68 69]
 [70 71 72 73 74 75 76 77 78 79]
 [80 81 82 83 84 85 86 87 88 89]
 [90 91 92 93 94 95 96 97 98 99]]
```

In [33]:
```
mat <  50 # it checks that element is less than 50 or not
```

Out[33]:
```
array([[ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [ True,  True,  True,  True,  True,  True,  True,  True,  True,
         True],
       [False, False, False, False, False, False, False, False, False,
        False],
       [False, False, False, False, False, False, False, False, False,
        False],
       [False, False, False, False, False, False, False, False, False,
        False],
       [False, False, False, False, False, False, False, False, False,
        False],
       [False, False, False, False, False, False, False, False, False,
        False]])
```

In [34]:
```
mat[mat<50] # if we apply mat twice it will consider as filter or masking
```

Out[34]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49])
```

In [35]:
```
mat[mat<=50]
```

Out[35]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50])
```

In [36]:
```
mat[mat>=50]
```

Out[36]:
```
array([50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66,
       67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83,
       84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

In [37]:
```
mat[mat>50]
```

Out[37]:
```
array([51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
       68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
       85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

In [38]:
```
mat[mat==50]
```

Out[38]:
```
array([50])
```

In [39]:
```
mat[mat!=50]
```

Out[39]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 51,
       52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
       69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85,
       86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99])
```

In [41]:
```
# List
a = [1,2,3]
b = [4,5,6]
result = [a[i] + b[i] for i in range(3)]
result
```

Out[41]:
```
[5, 7, 9]
```

In [42]:
```
# NumPy
import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])
result = a + b
result
```

Out[42]:
```
array([5, 7, 9])
```

In [72]:
```
ri_1
```

Out[72]:
```
array([[38, 45, 42, 31, 38, 48, 32, 36, 24, 41],
       [38, 41, 44, 30, 28, 40, 47, 23, 42, 30],
       [29, 30, 26, 42, 32, 30, 38, 46, 47, 23],
       [28, 42, 35, 24, 47, 28, 40, 35, 33, 42],
       [29, 42, 21, 30, 43, 31, 49, 35, 35, 42],
       [22, 36, 36, 28, 43, 28, 49, 22, 42, 46],
       [42, 45, 31, 25, 44, 20, 31, 41, 22, 31],
       [25, 36, 34, 41, 22, 21, 35, 37, 49, 44],
       [43, 29, 33, 28, 47, 39, 34, 43, 44, 20],
       [23, 43, 20, 23, 46, 29, 35, 24, 35, 34]], dtype=int32)
```

In [91]:
```
ri_1[:-3]
```

Out[91]:
```
array([[38, 45, 42, 31, 38, 48, 32, 36, 24, 41],
       [38, 41, 44, 30, 28, 40, 47, 23, 42, 30],
       [29, 30, 26, 42, 32, 30, 38, 46, 47, 23],
       [28, 42, 35, 24, 47, 28, 40, 35, 33, 42],
       [29, 42, 21, 30, 43, 31, 49, 35, 35, 42],
       [22, 36, 36, 28, 43, 28, 49, 22, 42, 46],
       [42, 45, 31, 25, 44, 20, 31, 41, 22, 31]], dtype=int32)
```

In [46]:
```
ri_1[0:3,0:3]
```

Out[46]:
```
array([[38, 45, 42],
       [38, 41, 44],
       [29, 30, 26]], dtype=int32)
```

In [47]:
```
ri_1[2:4,3:6]
```

Out[47]:
```
array([[42, 32, 30],
       [24, 47, 28]], dtype=int32)
```

LINSPACE #Linspace is aslo known as linear space

In [52]:
```
np.linspace(-5,5,3) # (lower range, upper range, number of elemetns printed)
```

Out[52]:
```
array([-5.,  0.,  5.])
```

In [56]:
```
np.linspace(2,5,3) #Linspace is aslo known as linear space
```

Out[56]:
```
array([2. , 3.5, 5. ])
```

In [ ]:
```
IDENTITY #Identity matrix represents that diagonals will be 1's and remaining ar
```

In [57]:
```
np.identity(2)
```

Out[57]:
```
array([[1., 0.],
       [0., 1.]])
```

In [71]:
```
z=np.identity(9)
print(z)
```

```
[[1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 1.]]
```

In [ ]:
```
NDIM (ndim) # It use used to find the dimensions of given array
```

In [60]:
```
mat1 = np.arange(0,9).reshape(3,3)
mat1
```

Out[60]:
```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

In [61]:
```
mat1.ndim
```

Out[61]:
```
2
```

In [62]:
```
mat2 = np.arange(0,100).reshape(25,4)
mat2
```

Out[62]:
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23],
       [24, 25, 26, 27],
       [28, 29, 30, 31],
       [32, 33, 34, 35],
       [36, 37, 38, 39],
       [40, 41, 42, 43],
       [44, 45, 46, 47],
       [48, 49, 50, 51],
       [52, 53, 54, 55],
       [56, 57, 58, 59],
       [60, 61, 62, 63],
       [64, 65, 66, 67],
       [68, 69, 70, 71],
       [72, 73, 74, 75],
       [76, 77, 78, 79],
       [80, 81, 82, 83],
       [84, 85, 86, 87],
       [88, 89, 90, 91],
       [92, 93, 94, 95],
       [96, 97, 98, 99]])
```

In [63]:
```
mat2.ndim #Here 2 is output because here row and columns are the dimensions
```

Out[63]:
```
2
```

In [65]:
```
mat3 = np.arange(0,5)
mat3
```

Out[65]:
```
array([0, 1, 2, 3, 4])
```

In [66]:
```
mat3.ndim #Here 1 is output because here row is the only dimension
```

Out[66]:
```
1
```

In [67]:
```
mat3
```

Out[67]:
```
array([0, 1, 2, 3, 4])
```

SIZE = NO OF ELEMENTS IN THE ARRAY (ROWS X COLUMNS)

In [92]:
```
mat3.size
```

Out[92]:
```
5
```

In [93]:
```
mat2.size
```

Out[93]:
```
100
```

In [ ]:
```
Data Type
```

In [95]:
```
print(mat2.dtype)
```

```
int64
```

In [96]:
```
mat1.dtype
```

Out[96]:
```
dtype('int64')
```

In [ ]:
```

```